VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	Registration Number
KHUSHBU SINGH	24BCE10214
Slot	**Date of Submission**
B22	31/03/2026

1. Introduction

Software that is freely developed and shared by the world's developer community powers the majority of the contemporary digital world. Many significant tools used in scientific computing, web development, artificial intelligence, and operating systems are developed collaboratively under open source principles rather than by a single business for financial gain. Open Source software promotes creativity, transparency, and community involvement by enabling anyone to view, alter, and distribute it without restriction.

The Python programming language is one of the most important open-source technologies in contemporary computing. Web development, data science, automation, machine learning, and system administration all make extensive use of Python. Its readability, simplicity, and robust community support are the reasons behind its popularity. Python and its libraries are being improved by thousands of developers worldwide.

The objective of this project is to perform an organized audit of the open-source Python software. Python's history and philosophy, the licensing model that controls its use, the moral ramifications of open source development, and its place in the Linux ecosystem are all examined in this report. Additionally, it compares Python with proprietary alternatives and looks at how Python works with other open-source tools.

The purpose of this report is to provide a detailed overview of what Python can do as a technology and why it exists, who creates and maintains it, and how open source principles have impacted the way in which it was developed. The report will demonstrate how being open source has shaped Python from both a functional and ideological standpoint, illustrating the important role open source plays in today's software development environment.

As future software engineers, you will need to understand the philosophical basis of open source software development. Many modern applications are developed using one or more open-source software components such as frameworks, libraries or tools — meaning that understanding how open-source projects operate, how different communities work together to contribute to the same goal; and how software licenses control the distribution and use of software are all key aspects of becoming an effective and able-bodied developer.

2. Chosen Software

Software	Category	License	Why it's interesting
Python	Programming Language	`PSF License`	*A language shaped entirely by community*

3. Part A — Origin and Philosophy

The Python language is a high-level interpreted programming language that was designed to be easy to write and read. Since Python supports the three basic programming paradigms for example imperative, object-oriented, and functional.

Since its creation by Guido Van Rossum in 1991, Python has seen tremendous growth with a large user base around the world. Python can be used in many different areas including web site development, artificial intelligence and machine learning development, automating repetitive tasks and writing scripts, analyzing numerical and graphical data, computing scientific data, and testing software.

A1. The problem Python was created to solve

The coding time for many programming languages that existed prior to the creation of Python was lengthy, because of how hard it was to create and read code. Coding in programming languages like C and C+ + and many others took lots of codes to accomplish even simple tasks and because of this it caused coding to be very difficult for beginners to learn and slowed down development.

At the time, Guido van Rossum was on the team at the Centrum Wiskunde & Informatica (CWI) research facility located in the Netherlands. He was interested in developing a list of characteristics for a new language, which would be easy to learn, easy to read, powerful enough to create a program from the language that could perform real world tasks and allow programmers to program without worrying about the complexity of the coding syntax. Allowing programmers to spend more time solving problems rather than coding complex syntax.

Guido was also inspired in designing Python from the ABC programming language in that he wanted to create something that incorporates all three of these characteristics while at the same time have the added characteristics of extensibility and practicality.

Another important concept to Guido in the design of Python was code readability. While all programming languages define code blocks or code sections using braces or brackets, Guido decided that in order for anyone who coded in Python to easily read and understand code, the indentation of code

lines the space before the beginning of the line of code would be used in place of braces, thus enhancing the readability of code in Python.

The Philosophy of Python is summed up in the Zen of Python, provided by Guido van Rossum, which states that the following three principles are core to the philosophy of Python which are Simplicity is better than complexity, Readability is important and There should be one and only one obvious way to accomplish a task.

The launch of Python as open-source software enabled developers globally to contribute enhancements, generate libraries, and broaden their ecosystems. This method of collaboration has contributed to Python’s significant growth and establishment as one of the leading programming languages utilized across multiple types of industries, including web development and artificial intelligence.

A2. The license — what it actually says

The Python Software Foundation provides the Python programming language under the Python Software Foundation's license which was approved as an Open Source Initiative. Under this license the usage of the Python programming language is completely free (both for commercial and private use). All users of the Python programming language are permitted to use their source code to create new programs based on the original source code of the Python programming language so long as they adhere to the guidelines of the Python Software Foundation's licensing agreement. In addition, the license also protects the original authors' rights and ensures that the programming language will be available to everyone to use.

The philosophy of Free Software is based on several key freedoms that users should have when using any type of software; a user should be able to run the software for any purpose, understand how it works, adapt or change it to meet their needs, and share their modified copy with others. Through its licensing, the Python Software Foundation provides support for free software users' ability to interact with and utilize the source code of the Python programming language under no restrictive limitations.

When it comes to licensing of software you will find that software licenses can be broken into two types permissive and restrictive licenses. There are some

licenses like the “General Public License”, “GPL”, when any software is modified, the entire software must still be made available to be considered as Open Source Software under GPL. This ensures that any improvements made to Open Source Software remain available to the community, while some other licenses such as the MIT License and Python Software Foundation License, are considered to be more permissively, allowing businesses to take the software and build proprietary products. Python License is also a permissive license and has allowed businesses and technology companies widespread use of Python.

When we talk about "Open Source Software" they may use 2 phrases to express what they mean, those 2 phrases are "Free As In Free Beer" and "Free As In Freedom." Free As In Free Beer means that there is no charge for the software; however there is no assurance that users may modify, distribute, or otherwise utilize this software for their own purposes - it may only be used as is. Free As In Freedom means that users have full access to the source code of the software and that users may use, study, modify, and otherwise use this software in whatever way they see fit. Python Software is a good representation of Freedom since it makes the complete source code available to developers to allow them to modify and adapt the code as necessary to meet their development needs.

A3. The ethics of open source

Open-source software raises ethical questions around transparency, accountability, and community responsibility. By design, open-source projects allow anyone to view, modify, and distribute code, which promotes collaboration and innovation. However, this openness also means maintainers have a duty to ensure their code is secure and well-documented. For example, vulnerabilities like the Heartbleed bug in OpenSSL highlighted how critical infrastructure relying on open-source tools can put users at risk if maintainers lack resources to audit code thoroughly. Ethical practice here involves prioritizing security updates and clearly communicating risks to users, even when projects are maintained voluntarily.

Another key consideration is inclusivity and fair contribution. Open-source communities often depend on unpaid labour, which can lead to burnout among maintainers and exclude contributors from underrepresented groups. Projects must actively foster respectful environments through codes of conduct and

ensure accessibility for non-native English speakers or developers with disabilities. For instance, projects like Django enforce documentation standards to lower barriers for new contributors. Ethically, maintainers should also credit contributors fairly and avoid exploiting unpaid work such as when companies profit from open-source tools without supporting their development.

Finally, licensing and sustainability pose ethical challenges. Choosing a license determines how others can use the software. For example, a permissive license might allow corporations to incorporate code into proprietary products without sharing improvements, raising questions about equity. Meanwhile, projects like OpenSSL faced sustainability crises when underfunded, leading to reliance on corporate sponsorships or donations. Ethically, developers should select licenses aligned with their values and advocate for sustainable funding models to ensure long-term maintenance without compromising accessibility. Balancing freedom, fairness, and practicality is central to ethical open-source development.

Open-source software raises ethical concerns related to sharing technology and knowledge. Additionally, some individuals argue that all forms of software development should be done in an open manner to promote transparency and allow anyone to review or check the source code for errors and defects. Furthermore, developing applications collaboratively allows programmers to improve upon each other's work, resulting in both faster innovation and greater reliability of software.

However, proponents of proprietary models argue that developers (organizations) invest not only time but also millions of dollars developing complicated software & hardware products, which they may choose not to share publicly in order to protect their investments, maintain their competitive edge, or avoid disclosing their proprietary source code to third parties. Support also comes from proponents of proprietary models since they provide dedicated customer support, maintenance services, etc., that some users may desire.

A related ethical question is whether companies that profit from the use of open-source software should compensate the people who created it. Many large technology organizations utilize open source software (such as Linux and Python) to produce revenue-generating products, leading critics to assert that these corporations benefit from the open-source community by utilizing its members' product development efforts without adequately compensating

them. Proponents argue that companies provide mutual benefit to the community by investing money and resources into their open-source projects.

Part B — Linux Footprint

Python is really important in Linux environments. It is one of the most popular programming languages used in different Linux systems. For example in Ubuntu Python is already installed because the operating system uses it for scripting and automation tasks. This shows that Python is not a tool for development but it is also a key part of how the system works.

If Python is not already installed you can easily install it using the Advanced Package Tool or APT for short with the command sudo install python3. This way you can be sure that Python is downloaded, verified and installed correctly along with all the things it needs to work

Once Python is installed it is put in directories in the Linux file system in a standard way. The main Python file is in /usr/bin/python3, which lets you run Python programs from the command line. This directory has files that can be used by the whole system. The standard libraries and modules that Python needs are in /usr/lib/python3, which has prewritten code that developers can use to do things like handle files work with networks and process data. There may also be some configuration files in directories like /etc. Python does not need much configuration compared to other software systems.

When it comes to managing users and groups Python works under the user who is currently logged in than using a special system account. This makes it more secure because Python processes get the permissions as the user who is running them. So Python cannot access system files without permission. This is how Linux works, where permissions and ownership are crucial for keeping the system safe.

Python is different from server software like web servers or databases because it does not run in the background all the time. Instead it works as an interpreter that runs scripts when the user tells it to. However Python applications can be set up to run as services using tools like systemd. For example a web application made with Python can be set up as a service that

starts automatically when the system boots, which shows how flexible Python can be.

The way Python gets updated in Linux systems is through package managers, like APT. The open-source community releases updates that fix bugs make Python work better and patch security issues. These updates are available everywhere. Can be installed easily using commands like sudo apt update and sudo apt upgrade. This way users can always have the version of Python and stay safe. Overall how Python works with Linux shows that open-source collaboration and standard system architecture can be very effective. Python is a part of Linux and it is used in many different ways.

Part C — The FOSS Ecosystem

Python is a part of a huge and constantly changing world of Free and Open Source Software. This world is made up of tools, libraries and frameworks that all work together to create powerful solutions. One of the things about Python is that it can work seamlessly with other technologies. This makes it a central part of software development environments. At its core Python relies on programming languages like C to function. Many of the parts of Python and the libraries that need to run fast are written in C. This allows Python to be easy to use and run efficiently at the same time.

Python also has a standard library that supports a lot of different functions. These libraries make it possible for developers to do tasks like working with files interacting with databases, networking and analyzing data without needing any extra tools. This built-in support saves developers time. Helps them be more productive. This is why Python is a choice for both beginners and experienced professionals.

Python has also made it possible to create frameworks and tools that add to its capabilities. For example Django is a framework that makes it easy to build scalable web applications. Flask is another framework that's lighter and gives developers more flexibility. In the field of intelligence and machine learning libraries like TensorFlow and PyTorch have become very popular. They allow developers to build models and applications. These tools show how Python is a

foundation for innovation in areas, including web development, data science, automation and artificial intelligence.

When it comes to web development Python plays a role in the backend. It is often used to handle the logic on the server side interact with databases and manage application workflows. Python works with web servers like Apache and Nginx. This makes it a reliable choice for building web applications that can scale and are easy to maintain. Python is also easy to read and understand, which makes it easy for developers to write efficient code. This is especially important for projects.

The community around Python is also very strong and active. Developers from over the world contribute to Python by creating libraries reporting issues and improving existing features. There are platforms where developers can collaborate and share knowledge, such as the official Python website, GitHub repositories and online forums. This community-driven approach ensures that Python is always improving and that problems are solved quickly. These are characteristics of successful open-source projects.

The Python Software Foundation is a -profit organization that manages the overall development and governance of Python. The foundation makes sure that Python stays source maintains its quality and supports its global community. Through conferences, documentation and community initiatives the foundation plays a role in keeping Pythons ecosystem healthy.

In the end Pythons place in the world of Free and Open Source Software shows the power of working and sharing knowledge. Its ability to work with technologies support many different applications and maintain a strong community makes it one of the most influential open-source projects in the world. Python is an example of what can be achieved when people work together towards a common goal. Python and its community are always moving forward. Making Python better. This is why Python is so important in the world of software development. Python will continue to be a part of the software development world, for a long time.

Part D — Open Source vs Proprietary

Dimension	Open Source	Proprietary Alternative
Cost	It is completely free to use, distribute, and modify. Does not have to pay licensing fees.	It is expensive licensing model. It Requires paid subscription.
Security (who can audit the code?)	It's Source code is fully open and publicly available to everyone. Anyone can inspect, audit, and improve it. Vulnerabilities are quickly identified by the community.	It is a Closed source. Users must rely entirely on the company for security updates and trust their internal processes.
Support and reliability	The Large global community support through forums, GitHub, Stack Overflow, and documentation and Commercial support is available via third party services.	The Official professional support with dedicated help, documentation, and guaranteed responses. Widely used in academia and industry.
Freedom to modify	It has full freedom under open-source license. Users can modify, redistribute, and even build their own versions from it.	No freedom is given to modify source code. Users are restricted to features provided by the company.
Community vs corporate control	It is community driven and maintained by the Python Software Foundation. No single company controls its future.	Completely controlled by MathWorks. All decisions, pricing, and updates depend on the company.
Your overall verdict	Python with libraries like NumPy, SciPy, Matplotlib, and TensorFlow Python has become a complete alternative to MATLAB	MATLAB provided powerful built-in capabilities, it also created vendor dependency. If a university or company stops paying for the license, access to the tool is lost immediately.

Verdict: Would I Recommend Python for Real-World Deployment?

Python is a choice for most real-world applications. It is often the practical option. The fact that Python is source means that anyone can use it from students to big companies without having to pay for it. There are thousands of libraries for Python, which makes it good for many different areas, such as making websites, artificial intelligence, data science, automation and more.

One thing that makes Python really strong is that it can be used in different ways. Whether you are working on a project for school or a big project for a company Python has the tools you need to get the job done. It also works well with technologies and platforms which makes it easy to use when building new software.

Python is not the best choice for everything. For some special jobs, like complicated math problems or systems that need to work in real time other tools like MATLAB might be better. MATLAB is really good at those kinds of jobs. Has a lot of support from professionals so it is often used in certain areas of engineering and research.

With its limitations Python is still a great choice. It is open has a community of users and is always getting better. This is why Python is so popular in the software industry. So I think Python is a choice, for most real-world applications especially when you need something that is flexible, affordable and can grow with your needs. Python is a choice because it is Python. I like Python because Python is easy to use and Python has libraries. Python is a language and Python is widely used.

Would I Contribute to Python?

Contributing to Python is an opportunity. It is also a challenge. When you look at open-source projects you can make contributions easily by sending in pull requests.. Python is different. The core development process for Python is more structured. People review it very carefully. When you make a contribution to Python it goes through a lot of discussions and code reviews and testing before it gets accepted.

This process is really thorough. That is what makes sure that Python is of high quality and stable. It also shows that Python is a project and that it is important to keep everything consistent in a language that so many people use.

If I wanted to contribute to Python I would start with things like documentation. Reporting bugs or testing. These are ways for new people to get started and they still help the community a lot. As I learned more about the system I could make complex contributions to Python over time.

Contributing to Python feels like I am giving back to something that helps me a lot and does not cost me anything. I do not have to contribute to Python. I feel like I should because I appreciate what it does for me. Open-source software is only good because a lot of people work together on it and even small contributions, to Python can make a difference.

Shell Script Tasks

Script 1 — System Identity Report

Purpose Statement This script makes a system identity report. It works like a screen for an audit project. It gets information about the Linux system. This includes the kernel version the user logged in now how long the system has been running, the date and details about the Linux distribution. The goal of this script is to give a look at the system being audited. This makes open-source systems more transparent and easy to check.

Shell Concepts Used The script uses all capital letters for variables. These store information, like the students name and software chosen. This makes the output look the same every time. The script also uses command substitution. This gets system information using commands like uname -r, whoami, uptime -p date and lsb_release. These commands put real-time system data into variables. The script shows how to format output using echo. It adds lines to make the output easier to read. This makes the script output look professional. The script uses shell scripting ideas. These are setting values running commands and formatting output.

code

```
#!/bin/bash


# Script 1: System Identity Report


# --- Variables ---
STUDENT_NAME="Khushbu Singh"
SOFTWARE="Python"


# --- System info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DATE=$(date)
DISTRO=$(lsb_release -d | cut -f2)


# --- Display ---
echo "================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "================================"
echo "Software : $SOFTWARE"
```

```
echo "Kernel   : $KERNEL"

echo "User     : $USER_NAME"

echo "Uptime   : $UPTIME"

echo "Date     : $DATE"

echo "Distro   : $DISTRO"

echo "License  : Open Source (GPL/Linux based)"
```

```
khushbu@LAPTOP-EG2AH51L
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ touch script1.sh script2.sh script3.sh script4.sh script5.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ nano script1.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ nano script3.sh

khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ nano script2.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ nano script4.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ nano script5.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ chmod +x script1.sh script2.sh script3.sh script4.sh script5.sh
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ ./script1.sh
================================
 Open Source Audit — Khushbu Singh
================================
Software : Python
Kernel   : 6.6.87.2-microsoft-standard-WSL2
User     : khushbu
Uptime   : up 1 hour, 18 minutes
Date     : Sun Mar 29 11:51:37 UTC 2026
Distro   : Ubuntu 24.04.4 LTS
License  : Open Source (GPL/Linux based)
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$
```

Script 2 — FOSS Package Inspector

Purpose Statement This script checks the system to see if Python is installed. It gets the version. Gives a short description of Python. The goal of this script is to show how Linux systems handle software and how users can check if something is installed and learn about software from the command line.

Shell Concepts Used The script uses if- to check if Python is installed. It uses dpkg -l and grep with a pipe to show how commands can work together. The script checks if Python is, in the systems list of packages. It also uses a command to get the version of Python. The script has a feature that prints different messages based on the package name. This shows how shell scripts can make decisions. The script also handles errors. Gives feedback to the user. It makes sure the user gets a message whether Python is installed or not. The script is an example of how to use the command line to manage software. It uses Python as an example to demonstrate these concepts.

code

```
#!/bin/bash


# Script 2: FOSS Package Inspector


PACKAGE="python3"


if dpkg -l | grep -q $PACKAGE; then
   echo "$PACKAGE is installed."
   python3 --version
else
   echo "$PACKAGE is NOT installed."
fi


case $PACKAGE in
   python3) echo "Python: a powerful open-source programming language" ;;
   git) echo "Git: version control system built for collaboration" ;;
   apache2) echo "Apache: web server that built the open internet" ;;
   mysql-server) echo "MySQL: open-source database system at the heart of millions of apps" ;;
   *) echo "Unknown package" ;;
Esac
```

khushbu@LAPTOP-EG2AH51L

```
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$ ./script2.sh
python3 is installed.
Python 3.12.3
Python: a powerful open-source programming language
khushbu@LAPTOP-EG2AH51U:~/oss-audit-24BCE10214/scripts$
```

Script 3 — Disk and Permission Auditor

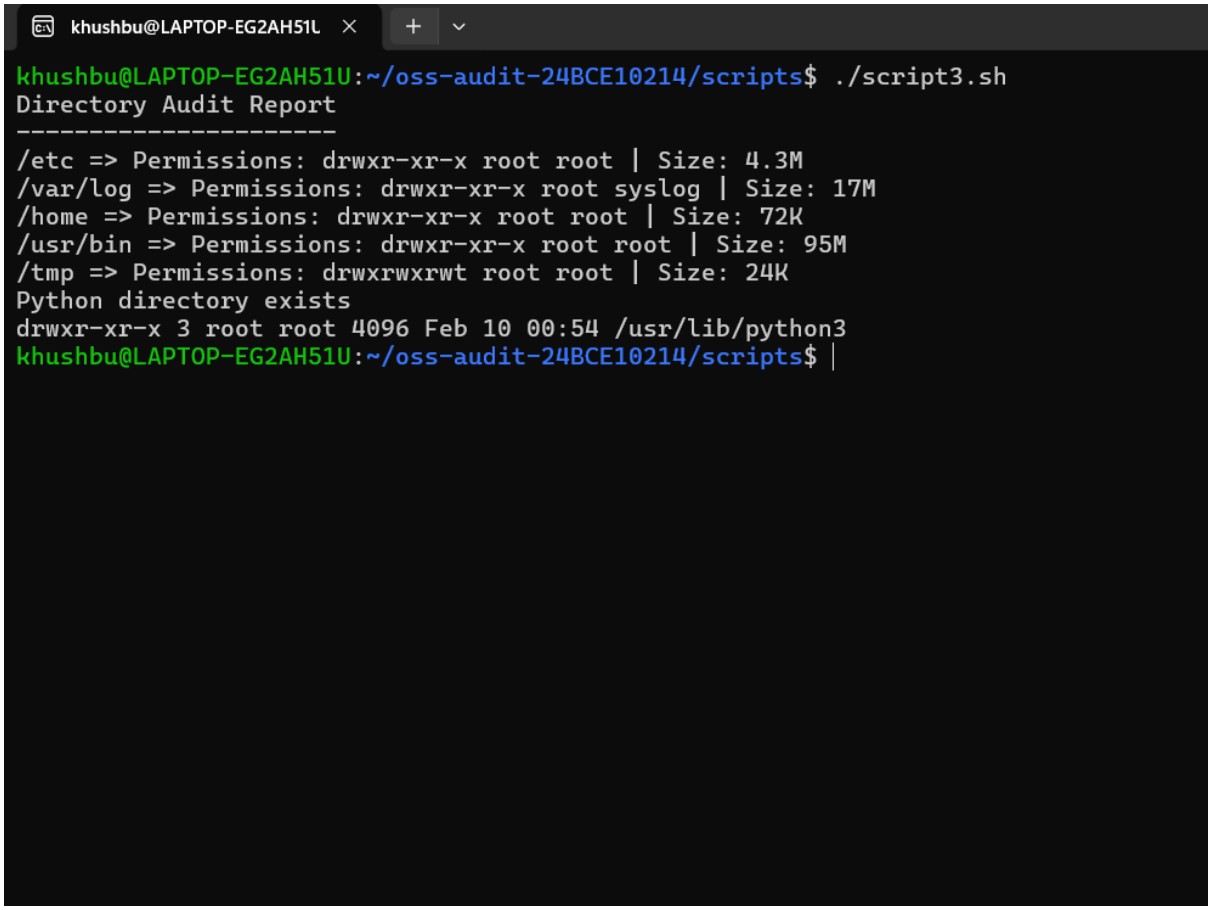
Purpose Statement This script does a system audit. It looks at directories and reports how much disk space they use, who owns them and their permission settings. This helps us understand how storage and access control work in a Linux system. It is essential for understanding system security and managing resources.

Shell Concepts Used The script uses a for loop to go through a list of directory paths. This shows how to automate tasks efficiently. It also checks if each directory exists before processing it. This makes the script more robust. To get permission details the script uses the ls -ld command. It then uses tools like awk to extract information such as permissions, owner and group. The script calculates disk usage with du -sh. It then refines the output using cut. This shows how to chain command-line utilities for data extraction. The script also has a check, for the Python directory. This links the audit directly to Python. Overall it demonstrates loops, conditionals and text parsing. These are components of effective shell scripting.

Code

```
#!/bin/bash


# Script 3: Disk and Permission Auditor


DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")


echo "Directory Audit Report"

echo "----------------------"


for DIR in "${DIRS[@]}"; do

  if [ -d "$DIR" ]; then

    PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')

    SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)

    echo "$DIR => Permissions: $PERMS | Size: $SIZE"

  else

    echo "$DIR does not exist"

  fi

done
```

```
# Check Python config directory exists or not

if [ -d "/usr/lib/python3" ]; then

    echo "Python directory exists"

    ls -ld /usr/lib/python3

fi
```



Script 4 — Log File Analyzer

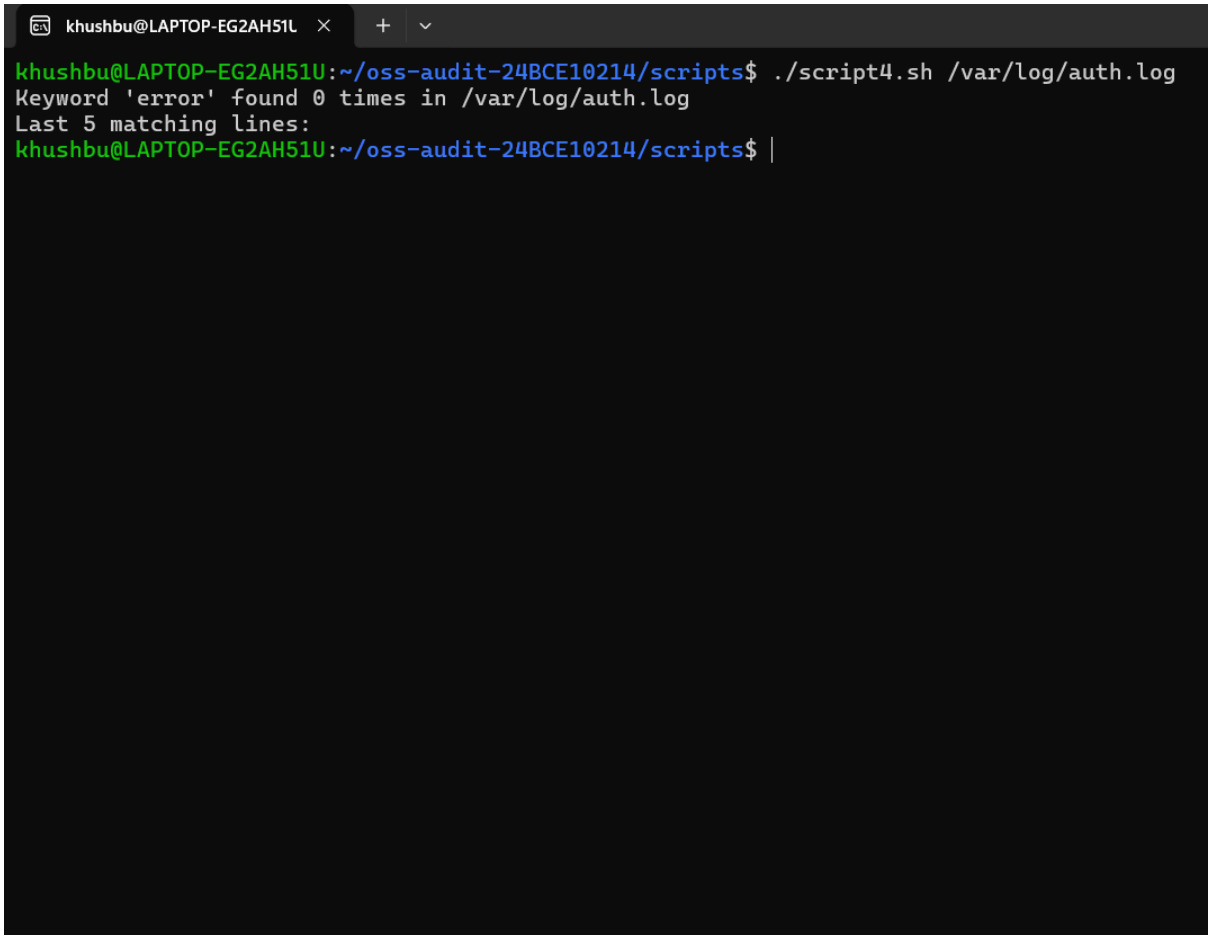
Purpose Statement This script looks at system log files to find and count how times a specific word, like "error" appears. It helps keep an eye on system health and find problems by checking log entries. The script also gives a summary of matching lines, which helps users quickly get an idea of whats happening with the system.

Shell Concepts Used The script shows how to use command-line arguments, like $1 and $2 so users can give the log file and keyword as input. It checks if the file exists before processing it using if statements to make sure everything is okay. The main part of the script reads the log file line, by line using a while loop. This is a way to handle big files. Inside the loop it uses grep to see if each line has the word and it doesn't matter if its uppercase or lowercase. The script keeps track of how times the word appears and it shows the last five lines that match using grep and tail together. This helps users see what’s going on with the system.

Code

```
#!/bin/bash


# Script 4: Log File Analyzer


LOGFILE=$1

KEYWORD=${2:-"error"}   # Default keyword is 'error'

COUNT=0


if [ ! -f "$LOGFILE" ]; then

   echo "Error: File $LOGFILE not found!"

   exit 1

fi


while IFS= read -r LINE; do

   if echo "$LINE" | grep -iq "$KEYWORD"; then

      COUNT=$((COUNT + 1))

   fi

done < "$LOGFILE"


echo "Keyword '$KEYWORD' found $COUNT times in $LOGFILE"
```

```
echo "Last 5 matching lines:"

grep -i "$KEYWORD" "$LOGFILE" | tail -5
```



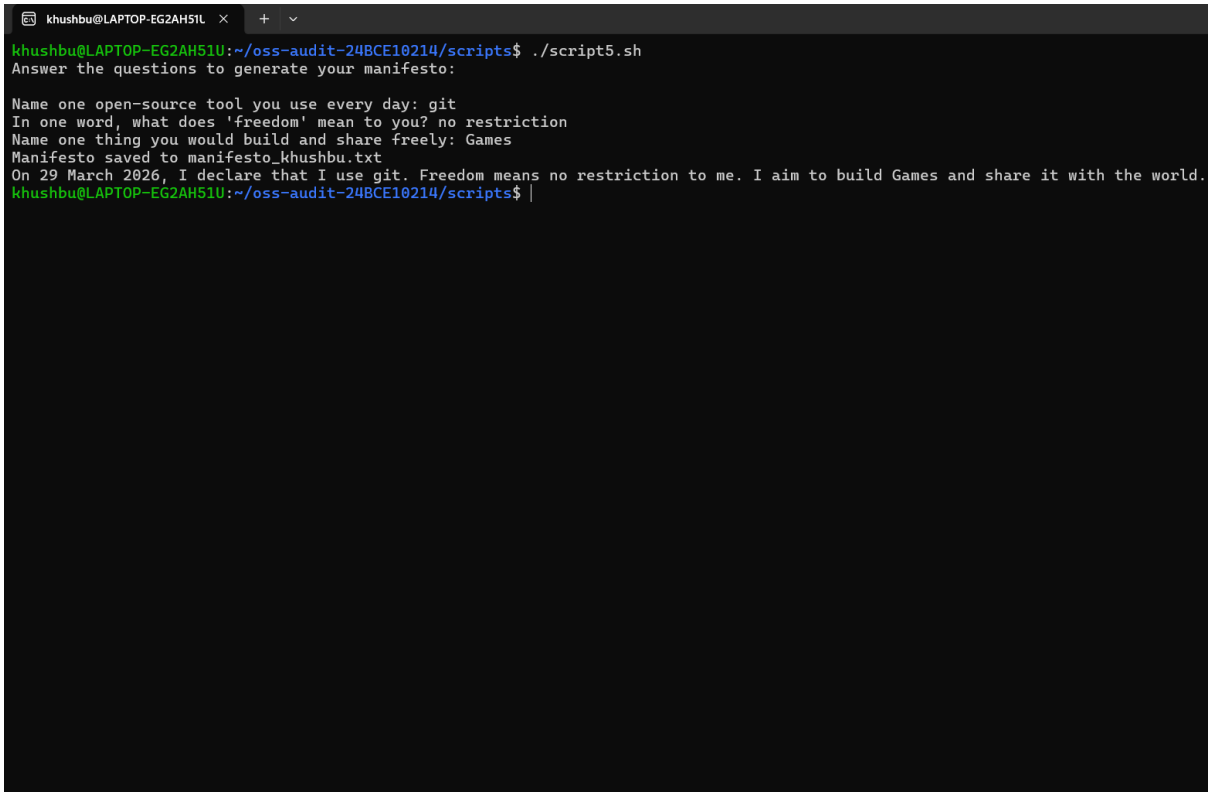
Script 5 — Open Source Manifesto Generator

Purpose Statement This script creates an open-source statement by collecting what you think and turning it into a structured statement. It focuses on the idea behind source by making you think about how you use it what freedom means to you and what you want to achieve by contributing.

Shell Concepts Used The script uses the read command to get your input making it interactive. It shows how to handle strings and substitute variables, where your inputs are stored and then put together to form a paragraph. The script also uses the date command to get todays date. Adds it to the output. It shows how to handle files by writing the generated statement to a text file. Finally the script uses the cat command to show you the files contents. This script is about interactivity working with files and string manipulation, in shell scripting.

Code

```
#!/bin/bash


# Script 5: Open Source Manifesto Generator


echo "Answer the questions to generate your manifesto:"

echo ""


read -p "Name one open-source tool you use every day: " TOOL

read -p "In one word, what does 'freedom' mean to you? " FREEDOM

read -p "Name one thing you would build and share freely: " BUILD


DATE=$(date '+%d %B %Y')

OUTPUT="manifesto_$(whoami).txt"


echo "On $DATE, I declare that I use $TOOL. Freedom means $FREEDOM to me. I aim to build $BUILD and
share it with the w>


echo "Manifesto saved to $OUTPUT"

cat $OUTPUT
```



Conclusion

This project has given me an understanding of open-source software. I learned about it from angles, not just the technical side, but also the philosophical and practical sides. I did an audit of Python and saw how open-source software is made, shared and improved by people working together all over the world. I looked at where Python came from its license and the ethical issues it raises. I realized that open source is not just about getting code for free but about being free, transparent and responsible.

When I looked at how Python works with Linux I got a lot of knowledge about how open-source software works with operating systems. I explored the directories, permissions and ways to install and update software. I understood how Linux systems are set up and how software is managed. This hands-on experience made the ideas I learned in theory much clearer and more meaningful.

The part of the project where I wrote shell scripts was really important for improving my skills. I wrote scripts to check the system verify packages look at disks, analyze logs and talk to users. I saw how automation works in real life. I also saw how small scripts can do things by combining basic Linux commands. These exercises helped me solve problems better and gave me confidence when working with command-line tools.

I also learned about the FOSS community and how connected modern software development is. Python is not alone it is part of a network of libraries, frameworks and

communities. This showed me that innovation in source comes from people working together not competing. Comparing source and proprietary software also showed me the trade-offs between being flexible costing money being in control and getting support.

Overall this project showed me how important open-source software is in todays world. It showed me that open source helps people learn, innovate and access things over the world. As a student and future developer this experience encouraged me to use open-source tools and think about contributing to them. Even small contributions can make a difference, to the community.

In conclusion this project combined. Practice, giving me a complete view of open-source software. It improved my skills and helped me appreciate how people work together in modern software development.